

RAPPORT DE CONDUITE D'UN PROJET IA

Portfolio conversationnel basé sur une architecture RAG



Ingénierie IA

Projet personnel déployé sur un serveur OVH

État du projet : solution finalisée, déployée et supervisée

Technologies principales

Angular	FastAPI	Chroma	Mistral Medium
Docker Compose	GitHub Actions	Prometheus / Grafana	Fluentd / Loki

Synthèse du projet

Le projet consiste à intégrer à un portfolio public un chatbot reposant sur une architecture RAG (Retrieval-Augmented Generation). Il permet à un recruteur d'interroger les expériences professionnelles, les informations personnelles rendues publiques et les projets IA réalisés pendant la formation. La solution est finalisée, conteneurisée, déployée sur un serveur OVH et dotée d'un dispositif de supervision et d'évaluation.

Positionnement du projet

Il s'agit d'un prototype avancé et exploitable en conditions réelles, avec déploiement automatisé, limitation de débit, observabilité et évaluation RAGAS. Les principaux axes d'amélioration concernent désormais la protection des données personnelles, l'accès par nom de domaine et l'industrialisation des tests de non-régression.

Périmètre fonctionnel

- Répondre aux questions d'un recruteur à partir des informations présentes dans le CSV.
- Présenter les expériences, compétences et projets IA de manière conversationnelle.
- Limiter les réponses au contexte récupéré afin de réduire les hallucinations.
- Mesurer la qualité du RAG avec RAGAS et superviser l'application en production.

Architecture en une vue

Couche	Composants	Rôle principal
Interface	Angular	Interface publique destinée aux recruteurs.
API et orchestration	FastAPI	Validation des requêtes, recherche documentaire, appel au LLM et exposition des métriques.
RAG	Chroma, all-MiniLM-L6-v2, mmarco-mMiniLMv2-L12-H384-v1	Indexation, recherche vectorielle et reranking des passages.
Génération	Mistral Medium	Production de la réponse à partir du contexte sélectionné.
Exploitation	Docker Compose, GitHub Actions, OVH	Conteneurisation, déploiement et hébergement.
Observabilité	Prometheus, Grafana, Fluentd, Loki	Collecte des métriques, visualisation et centralisation des logs.

1. Contexte et analyse des besoins

1.1 Présentation et finalité

Le porteur du projet recherche un emploi dans le domaine de l'intelligence artificielle, en particulier autour des LLM et de l'automatisation. Le profil associe une formation en IA à des années d'expérience en support informatique. Le portfolio conversationnel vise à différencier la candidature en démontrant, par un cas d'usage concret, la capacité à concevoir, déployer et exploiter une solution IA complète.

1.2 Parties prenantes et besoin métier

Partie prenante	Attente	Réponse apportée
Recruteur	Obtenir rapidement des informations fiables et ciblées sur le candidat.	Interface simple, réponses courtes et fondées sur les données du portfolio.

Partie prenante	Attente	Réponse apportée
Candidat / développeur	Valoriser ses compétences IA, data, backend, frontend et DevOps.	Projet transversal démontrant l'ensemble du cycle de vie d'une application RAG.

1.3 Contraintes et priorités

Priorité	Besoin ou contrainte	Critère de satisfaction
Haute	Pertinence et fidélité des réponses	Les réponses s'appuient sur le contexte récupéré et atteignent les seuils RAGAS définis.
Haute	Protection des données personnelles	Les données publiées sont limitées au strict nécessaire ; salaire demandé, et adresse précise ne doivent pas être exposés sans justification.
Haute	Disponibilité et maîtrise des abus	L'API reste stable et le rate limiting réduit le spam et la consommation excessive du LLM.
Moyenne	Rapidité de réponse	La latence reste compatible avec une consultation de portfolio

Choix d'accès public

L'absence d'authentification est cohérente avec la fonction d'un portfolio : un recruteur doit pouvoir le consulter sans friction. Le risque principal n'est donc pas l'accès anonyme lui-même, mais la nature des données rendues accessibles publiquement et leur possible utilisation.

2. Audit de la solution data et IA existante

2.1 Données et préparation

La source de connaissance est un fichier CSV unique contenant les informations personnelles choisies pour le portfolio ainsi que les projets IA réalisés pendant la formation. Au démarrage ou lors d'une mise à jour, les contenus sont transformés en vecteurs puis enregistrés dans Chroma. Ce choix est adapté au volume limité du projet et simplifie la maintenance.

Élément	État actuel	Analyse
Source	Un fichier CSV statique	Simple à versionner et à contrôler, mais les mises à jour sont manuelles.
Embeddings	all-MiniLM-L6-v2	Modèle local léger, adapté à un corpus court et à un budget réduit.
Reranking	mmarco-mMiniLMv2-L12-H384-v1	Reclasse les résultats issus de la recherche vectorielle afin d'améliorer la précision du contexte fourni au LLM.
Base vectorielle	Chroma	Cohérente avec un déploiement local-first et un faible volume documentaire.
LLM	Mistral Medium	Retenu après abandon de Mistral Small, jugé insuffisant pour la qualité attendue.

2.2 Flux de traitement

1	Chargement et contrôle Lecture du CSV et préparation des contenus destinés à l'indexation.
2	Indexation Génération des embeddings avec all-MiniLM-L6-v2 et stockage dans Chroma.
3	Recherche Transformation de la question en vecteur et récupération des passages les plus proches.
4	Reranking Reclassement des passages avec mmarco-mMiniLMv2-L12-H384-v1.
5	Génération Construction du contexte et appel à Mistral Medium pour produire la réponse.
6	Traçabilité Production de logs et de métriques exploités dans Loki, Prometheus et Grafana.

2.3 Adéquation de la solution

Points forts	Limites et points de vigilance
Architecture compréhensible et adaptée au volume du corpus.	Le CSV statique nécessite une mise à jour manuelle et un contrôle de version des données.
Embeddings locaux et Chroma limitant les coûts récurrents.	L'adresse précise et le téléphone peuvent être exposés ou restitués par le chatbot s'ils restent dans le corpus public.
Reranker améliorant la sélection du contexte.	Les résultats d'évaluation dépendent du jeu de questions de test ; il doit rester représentatif des usages recruteurs.
Mistral Medium offrant une qualité supérieure au modèle initial.	Le coût et les quotas de l'API LLM doivent continuer à être surveillés.

Points forts	Limites et points de vigilance
Observabilité et rate limiting déjà opérationnels.	L'accès par adresse IP est moins lisible et moins professionnel qu'un nom de domaine.

3. Solution technique retenue

3.1 Justification des choix

Choix	Justification
Angular	Permet de construire une interface structurée, réactive et cohérente avec les compétences frontend visées.
FastAPI	Framework Python léger, performant et adapté à l'exposition de services IA et de métriques.
Chroma	Répond au besoin de simplicité, de persistance locale et de faible volumétrie sans introduire une infrastructure disproportionnée.
all-MiniLM-L6-v2	Fournit des embeddings compacts et rapides, sans coût d'API supplémentaire.
Reranker mmarco-mMiniLMv2-L12-H384-v1	Améliore l'ordre des passages récupérés avant l'appel au LLM.
Mistral Medium	Offre le compromis qualité / coût retenu après comparaison avec Mistral Small.
Docker Compose	Garantit un environnement reproductible et orchestre les services applicatifs et de supervision.
GitHub Actions	Automatise les contrôles et le déploiement, réduisant les opérations manuelles.
OVH	Fournit l'hébergement permanent de la solution avec un niveau de contrôle adapté à un projet personnel.

3.2 Architecture d'exploitation

Domaine	Composants	Fonction
Application	Angular + FastAPI	Interaction utilisateur et orchestration du pipeline RAG.
Données IA	CSV + Chroma	Source de connaissance et index vectoriel persistant.
Inférence	Embeddings + reranker + Mistral Medium	Recherche, sélection du contexte et génération de la réponse.
Protection	Validation des entrées + rate limiting	Réduction des requêtes abusives et des entrées non conformes.
Métriques	Prometheus + Grafana	Collecte et visualisation de la latence, de la disponibilité et de l'activité.
Logs	Fluentd + Loki	Collecte, centralisation et consultation des journaux applicatifs.
Livraison	GitHub Actions + Docker Compose + serveur OVH	Construction, déploiement et exécution reproductibles.

4. Stratégie de mise en œuvre

4.1 Démarche projet

Phase	Travaux réalisés	Livrable
1. Cadrage	Définition du besoin recruteur, du corpus et des contraintes de coût.	Périmètre fonctionnel et choix du RAG.
2. Construction du RAG	Préparation du CSV, embeddings, Chroma, recherche et prompt.	Pipeline de recherche et de génération.
3. Amélioration de la qualité	Passage de Mistral Small à Mistral Medium et ajout du reranker.	Réponses plus pertinentes et contexte mieux classé.
4. Intégration	Développement de l'API FastAPI et de l'interface Angular.	Application complète utilisable par un recruteur.
5. Industrialisation	Docker Compose, GitHub Actions, rate limiting et déploiement OVH.	Environnement reproductible et solution en ligne.
6. Exploitation	Prometheus, Grafana, Fluentd, Loki et évaluations RAGAS.	Suivi technique et contrôle de la qualité IA.

4.2 Intégration et déploiement

Docker Compose regroupe les services nécessaires au fonctionnement de l'application et de son observabilité. GitHub Actions automatise les étapes prévues dans le dépôt, puis la solution est exécutée sur le serveur OVH. L'accès actuel par adresse IP permet la mise en ligne, mais un nom de domaine constituera une amélioration de présentation.

4.3 Principaux risques et mesures

Risque	Impact	Mesure en place	Action complémentaire
Hallucination ou réponse non fondée	Information incorrecte donnée au recruteur.	Contexte RAG, prompt contraint, reranking et mesure de la faithfulness.	Ajouter les évaluations RAGAS aux tests de non-régression.
Exposition de données personnelles	Réutilisation indésirable de l'adresse ou du téléphone.	Données contrôlées dans un CSV unique.	Appliquer la minimisation : retirer l'adresse précise et limiter l'exposition du téléphone.
Spam ou consommation excessive	Dégradation du service et hausse des coûts.	Rate limiting.	Ajouter des alertes sur les volumes et les erreurs.
Indisponibilité ou latence élevée	Mauvaise expérience recruteur.	Prometheus, Grafana et logs centralisés.	Définir des seuils d'alerte opérationnels.
Régression après mise à jour	Baisse silencieuse de la qualité des réponses.	Jeu d'évaluation RAGAS.	Exécuter automatiquement le jeu d'évaluation dans la CI.

5. Évaluation et aide à la décision

5.1 Dispositif RAGAS

La qualité du système est contrôlée avec RAGAS. Des seuils d'acceptation ont été définis pour quatre métriques. Les résultats ci-dessous sont tous supérieurs aux seuils retenus sur le jeu d'évaluation utilisé.

Métrique	Résultat	Seuil	Statut	Interprétation
Faithfulness	0,844	0,72	Conforme	La réponse reste globalement fidèle au contexte fourni.
Answer relevancy	0,437	0,30	Conforme	La réponse traite la question, avec une marge d'amélioration encore possible.
LLM context precision with reference	0,894	0,70	Conforme	Les passages sélectionnés sont très majoritairement utiles à la réponse attendue.
Context recall	0,616	0,50	Conforme	Le contexte récupère une part suffisante des informations attendues.

Lecture des résultats

La meilleure performance concerne la précision du contexte (0,894), ce qui confirme l'intérêt du mécanisme de recherche complété par le reranker. La faithfulness est également solide (0,844). L'answer relevancy reste conforme mais constitue l'indicateur le plus proche de son seuil ; il doit être surveillé lors des évolutions du prompt, du corpus ou du modèle.

5.2 Indicateurs de pilotage

Axe	Indicateur	Source	Utilité
Qualité IA	Faithfulness, answer relevancy, context precision, context recall	RAGAS	Détecter les régressions du pipeline RAG.
Performance	Latence de l'API et du LLM	Prometheus / Grafana	Vérifier que l'expérience reste fluide.
Fiabilité	Taux d'erreurs et disponibilité des services	Prometheus / Grafana	Identifier rapidement une panne ou une dégradation.
Usage	Nombre de requêtes et déclenchements du rate limiting	Métriques et logs	Suivre l'activité et repérer les abus.
Diagnostic	Erreurs applicatives et événements techniques	Fluentd / Loki	Analyser la cause d'un incident.

6. Contrôle et suivi du projet

6.1 Pilotage opérationnel

Le projet a été conduit de manière itérative : chaque évolution technique a été validée par des essais fonctionnels, puis complétée par des mécanismes de supervision et d'évaluation. Le suivi actuel permet de couvrir à la fois le fonctionnement de l'application et la qualité du système RAG.

Contrôle	Fréquence recommandée	Décision associée
Consultation des tableaux Grafana	Régulière et après chaque déploiement	Vérifier la stabilité, la latence et les erreurs.
Analyse des logs Loki	En cas d'incident ou d'anomalie	Identifier le composant ou la requête à l'origine du problème.
Exécution du jeu RAGAS	Avant une mise en production modifiant le corpus, le prompt, le reranker ou le LLM	Bloquer une version si une métrique passe sous son seuil.
Revue du CSV public	À chaque mise à jour des informations personnelles	Éviter la publication de données non nécessaires ou trop précises.
Contrôle du rate limiting	Après une hausse d'usage ou un abus détecté	Ajuster les limites sans dégrader l'accès normal des recruteurs.

6.2 Critères de réussite

- Les quatre métriques RAGAS restent au-dessus de leurs seuils d'acceptation.
- Les réponses sont compréhensibles, utiles à un recruteur et limitées aux informations du corpus.
- L'application reste accessible et les incidents peuvent être diagnostiqués à partir des métriques et des logs.
- Le coût d'usage du LLM reste compatible avec le budget du projet personnel.
- Aucune donnée personnelle plus précise que nécessaire n'est exposée publiquement.

7. Conclusion

Le projet répond à son objectif principal : proposer un portfolio différenciant tout en démontrant la capacité à construire un système RAG de bout en bout. La solution associe une interface Angular, une API FastAPI, Chroma, des embeddings locaux, un reranker, langchain, Mistral Medium, un déploiement conteneurisé et une chaîne complète d'observabilité. L'ajout de RAGAS apporte une mesure objective de la qualité.

Fin du rapport